CÁC LUỒNG VÀO/RA DỮ LIỆU (INPUT – OUTPUT)

ThS. Trương Bá Vĩnh

Mục tiêu

1. Hiểu khái niệm về dòng.
2. Biết các lớp đảm nhiệm việc việc xuất nhập dữ liệu trong Java.
3. Biết cách sử dụng các lớp io để xuất nhập dữ liệu với màn hình và bàn phím.
4. Biết cách xuất nhập dữ liệu với tập tin văn bản, tập tin chứa các dữ liệu thuộc kiểu cơ bản, tập tin chứa dữ liệu là các đối tượng.
5. Biết các interface và các lớp quản lý việc in ấn.

Nội dung

3.1- Giới thiệu.

3.2- Dòng dữ liệu.

3.3- Gói java.io và các dòng nhập xuất

3.4- Lớp System và thiết bị xuất nhập chuẩn

3.5 – Ví dụ xuất nhập dữ liệu với tập tin văn bản

3.6- Tóm tắt

3.1- Giới thiệu

- Nhập dữ liệu là tác vụ đưa các dữ liệu cụ thể vào cho biến trong chương trình. Như vậy, phải có một nguồn chứa dữ liệu (bàn phím, tập tin, biến khác).
- Xuất dữ liệu là tác vụ đưa trị cụ thể của biến trong chương trình ra một nơi chứa (màn hình hay file hay biến khác).
- Nhập/xuất dữ liệu là các phương tiện mà chương trình tương tác với user và thường không thể thiếu trong đa số các ứng dụng.

Giới thiệu ...

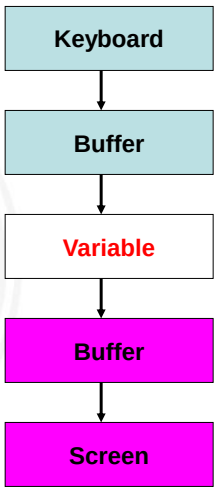
- Hai cơ chế nhập xuất dữ liệu có tương tác với user:

(1) Nhập xuất dữ liệu trong các ứng dụng console application,

(2) Nhập/xuất dữ liệu thông qua các phần tử trên GUI. Cách 1 thường dùng trong các ứng dụng chạy theo cơ chế tuần tự còn cách 2 được áp dụng trong các ứng dụng hướng cửa sổ.

Giới thiệu

- Buffered IO : Nhập xuất thông qua bộ đệm (một vùng vùng nhớ trung gian
- Nhập có đệm (buffered Input) : Dữ liệu nhập được đệm lại không đi vào biến ngay lập tức. ví dụ: Nhập 1 số chỉ kết thúc khi gõ enter.
- Xuất có đệm (buffered output): Dữ liệu xuất chỉ được xuất thực sự khi bộ đệm đầy hoặc khi gặp một tác vụ buộc xuất tường minh (flush).



Giới thiệu

- Tập tin là một dữ liệu mô tả cho một thông tin đã hoàn tất. Do vậy, tập tin có thể là dữ liệu đầu vào của chương trình và cũng có thể là nơi chứa dữ liệu đầu ra của chương trình. Hầu hết những chương trình lớn đều có thao tác với tập tin.
- **Khi nhập xuất dữ liệu có thể gây ra lỗi Exception**. ví dụ: Nhập biến số mà gõ chữ, đọc file vào biến mà vị trí đọc là cuối file, ghi file mà đĩa đã hết dung lượng,… Như vậy, khi xuất nhập dữ liệu, người lập trình cần có biện pháp quản lý các lỗi xuất nhập bằng cú pháp

```
try {
    TácVụNhậpXuất();  }
catch (Exception e)
  { System.out.println("Error: " + e.toString()); }
```

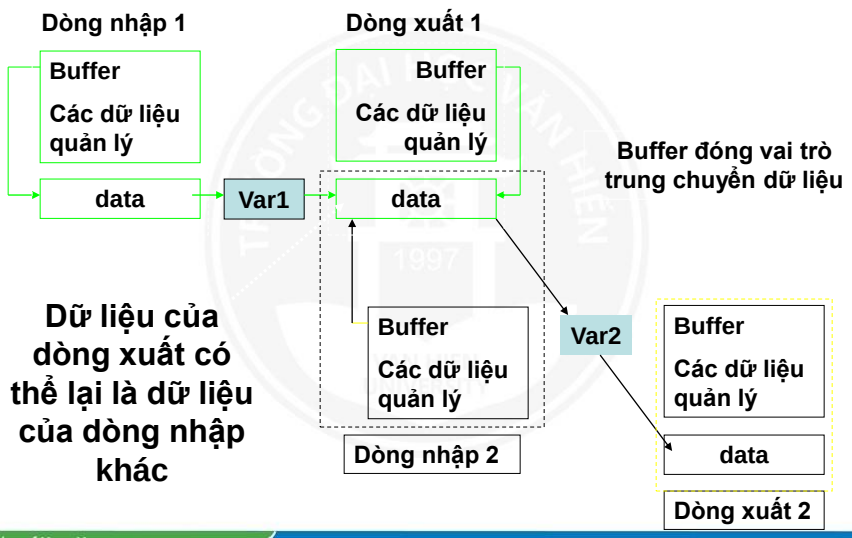

3.2- Dòng- stream

- Dòng: Là một chuỗi các byte làm việc theo cơ chế tuần tự.
- Khaí niệm dòng xuất phát từ hệ điều hành UNIX.
- Bàn phím là dòng nhập chuẩn, user gõ tuần tự các phím → chuỗi các byte đi vào biến.
- Màn hình là dòng xuất chuẩn, dữ liệu từ biến được chuyển thành các ký tự, ký số rồi các byte này lần lượt được xuất ra màn hình.

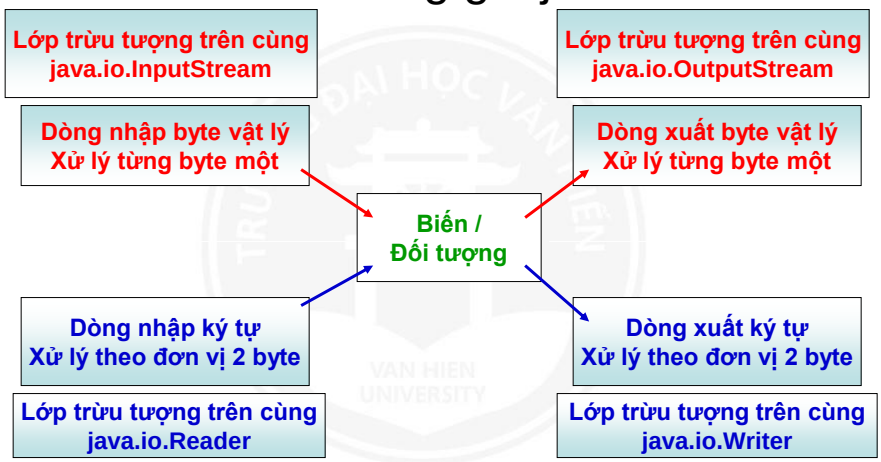
Chuỗi, mảng, file đều là dòng...

- Chuỗi ký tự, mảng các byte chứa dữ liệu được chuyển vào cho biến cũng làm việc theo cơ chế chuyển từng byte → Chuỗi, mảng dòng nhập. Ngược lại, có thể đưa dữ liệu từ biến ra chuỗi, mảng → Chuỗi, mảng trở thành dòng xuất.
- File cũng là nguồn cung cấp dữ liệu cho biến (file nhập), và cũng là nơi lưu trữ dữ liệu từ biến (file xuất). File làm việc theo cơ chế từng byte một → File là dòng.

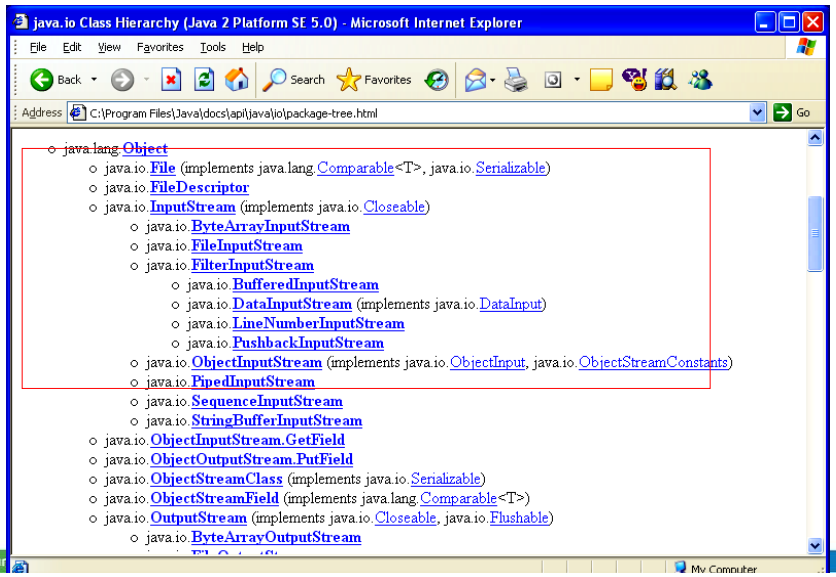
Buffer của dòng : mảng lưu trữ dữ liệu



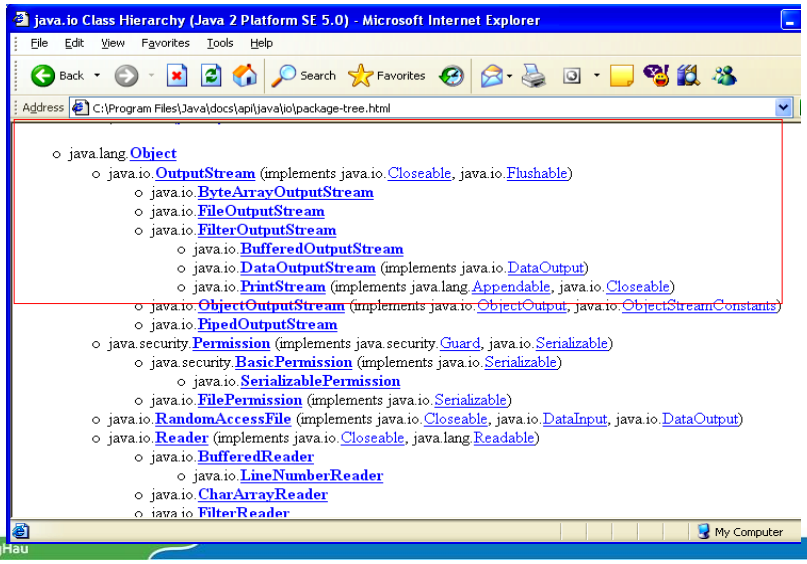
3.3- IO classes trong gói java.io



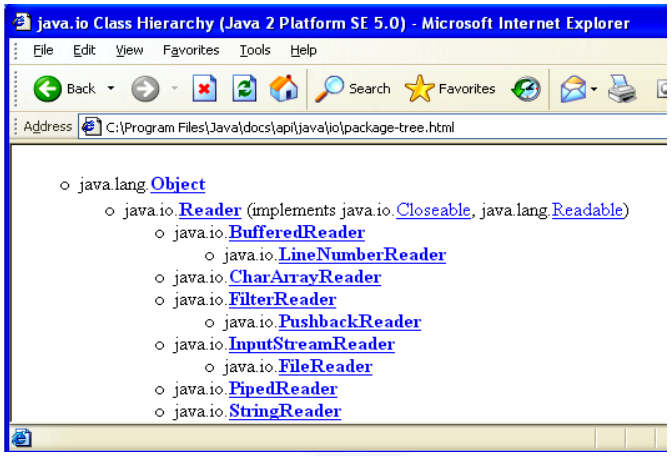
Phân cấp các lớp nhập theo byte vật lý



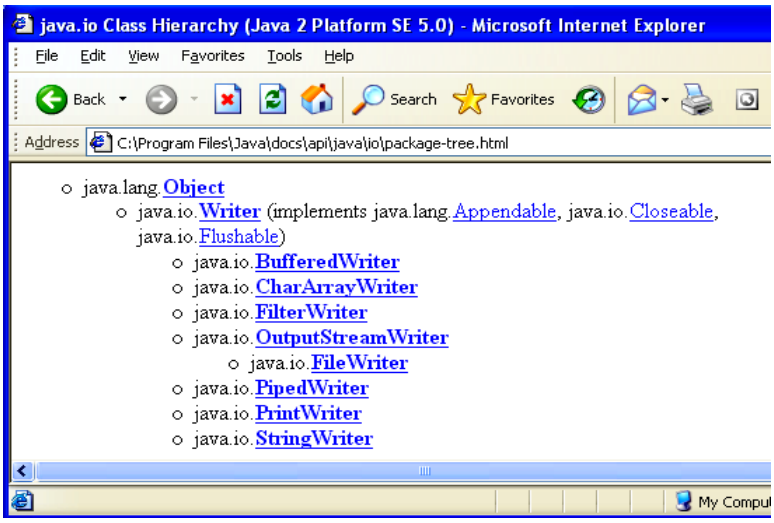
Phân cấp các lớp xuất theo byte vật lý



Phân cấp các lớp nhập theo ký tự



Phân cấp các lớp xuất theo ký tự



Phân cấp các lớp thao tác file với hệ điều hành

- java.lang.Object
  - java.io.File (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.io.FileDescriptor
  - java.io.RandomAccessFile (implements java.io.Closeable, java.io.DataInput, java.io.DataOutput)

- **Lớp File giúp truy xuất các thuộc tính của 1 file/thư mục.**
- **Lớp FileDescriptor: Giúp đồng bộ việc truy xuất file.**
- **Lớp RandomAccessFile giúp đọc/ghi file với dữ liệu thuộc kiểu cơ bản**

Các interface được khai báo trong java.io

3.3.1- Các dòng trừu tượng byte-vật lý InputStream/OutputStream

- Là hai lớp trừu tượng định nghĩa những thao tác truy xuất dữ liệu cơ bản (mức khái quát) theo từng byte vật lý mà không phân biệt nguồn dữ liệu là loại gì (file, chuỗi,…).
- Các lớp dẫn xuất từ hai lớp này nhằm cụ thể hóa các dòng nhập xuất byte vật lý tùy từng tình huống.

java.io.InputStream (implements java.io.Closeable)

java.io.OutputStream (implements java.io.Closeable, java.io.Flushable)

Abstract class InputStream Method Summary

constructor	InputStream()
int	available() : Trả về số bytes còn có thể được đọc (hay đã bị bỏ qua)
void	close() : Đóng dòng, trả tài nguyên đã được liên kết với dòng
void	mark(int readlimit) – Đánh dấu vị trí hiện hành, readlimit là số byte có thể được đọc trước khi vị trí đánh dấu không còn hợp lệ
boolean	markSupported() - Kiểm tra dòng có cho phép đánh dấu và reset không?
abstract int	read() Đọc 1 byte kế tiếp từ dòng
int	read(byte[] b) : Đọc dòng ra mảng các byte
int	read(byte[] b, int off, int len) Đọc len bytes từ dòng lưu vào mảng b từ phần tử thứ off
void	reset() Quay trở lại vị trí vừa được đánh dấu do method mark
long	skip(long n) Bỏ qua n bytes dữ liệu từ dòng

Abstract OutputStream Method Summary

constructor	OutputStream()
void	close() – đóng dòng xuất, trả tài nguyên
void	flush() Cưỡng bức ghi dữ liệu vào dòng xuất
void	write(byte[] b) : Ghi mảng các byte lên dòng
void	write(byte[] b, int off, int len) Ghi len byte từ phần tử thứ off của mảng lên dòng xuất
abstract void	write(int b) Ghi 1 byte vào dòng xuất

3.3.2- Lớp ByteArrayInputStream
Lớp ByteArrayOutputStream

- Là hai lớp con tương ứng của InputStream và OutputStream.
- Là hai dòng xuất nhập dạng mảng các bytes.
- Dữ liệu của lớp ByteArrayInputStream:
  byte [] buf : mảng các byte dữ liệu, **int count** : số byte hiện có
  int mark: vị trí đánh dấu hiện hành, **int pos**: vị trí hiện hành.
- Dữ liệu của lớp ByteArrayOutputStream:
  byte [] **buf** : mảng các byte dữ liệu, int **count** : số byte hiện có

Lớp ByteArrayInputStream
Lớp ByteArrayOutputStream

- **Constructors**

ByteArrayInputStream(byte[] buf)
Tạo 1 a ByteArrayInputStream với bộ đệm đã có.

ByteArrayInputStream(byte[] buf, int offset, int length)
Tạo 1 a ByteArrayInputStream với bộ đệm đã có kể từ vị trí offset với kích thức length.

ByteArrayOutputStream()
Tạo 1 mảng mới làm vai trò output stream.

ByteArrayOutputStream(int size)
Tạo 1 mảng mới làm vai trò output stream với kích thước size bytes

ByteArrayInputStream methods

- Các hành vi của lớp cha InputStream được cụ thể hóa.

int	**available**() Returns the number of bytes that can be read from this input stream without blocking.
void	**close**() Closing a ByteArrayInputStream has no effect.
void	**mark**(int readAheadLimit) Set the current marked position in the stream.
boolean	**markSupported**() Tests if this InputStream supports mark/reset.
int	**read**() Reads the next byte of data from this input stream.
int	**read**(byte[] b, int off, int len) Reads up to len bytes of data into an array of bytes from this input stream.
void	**reset**() Resets the buffer to the marked position.
long	**skip**(long n) Skips n bytes of input from this input stream.

Methods inherited from class java.io.InputStream
read

ByteArrayOutputStream methods

- Các hành vi của lớp cha OutputStream được cụ thể hóa.

void	**close**() Closing a ByteArrayOutputStream has no effect.
void	**reset**() Resets the count field of this byte array output stream to zero, so that all currently accumulated output in the output stream is discarded.
int	**size**() Returns the current size of the buffer.
byte[]	**toByteArray**() Creates a newly allocated byte array.
String	**toString**() Converts the buffer's contents into a string, translating bytes into characters according to the platform's default character encoding.
String	**toString**(int hibyte) **Deprecated.** *This method does not properly convert bytes into characters. As of JDK 1.1, the preferred way to do this is via the toString(String enc) method, which takes an encoding-name argument, or the toString() method, which uses the platform's default character encoding.*
String	**toString**(String enc) Converts the buffer's contents into a string, translating bytes into characters according to the specified character encoding.
void	**write**(byte[] b, int off, int len) Writes len bytes from the specified byte array starting at offset off to this byte array output stream.
void	**write**(int b) Writes the specified byte to this byte array output stream.
void	**writeTo**(OutputStream out) Writes the complete contents of this byte array output stream to the specified output stream argument, as if by calling the output stream's write method using out.write(buf, 0, count).

Methods inherited from class java.io.OutputStream
flush, write

Ví dụ về ByteArray Input/Output Stream

- Có dữ liệu

String S = "Ve ve ve, mua he sang";

- Chương trình sẽ ghi chuỗi này lên 1 ByteArrayOutputStream, sau đó lấy buffer của ByteArrayOutputStream chuyển sang 1 ByteArrayInputStream. Đọc từ ByteArrayInputStream ra biến, xuất biến.
- Chương trình cũng minh họa việc truy xuất kích thước buffer

Ví dụ về ByteArray Input/Output Stream

```
//ByteArrayDemo.java
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.lang.System;
class ByteArrayDemo
{ public static void main(String args[])
  { String S = "Ve ve ve, mua he sang";
    ByteArrayOutputStream outStream = new ByteArrayOutputStream();
    int i;
    // Ghi các ký tự từ chuỗi S lên outStream
    for (i=0;i<S.length(); ++i) outStream.write(S.charAt(i));
    System.out.println("Dong xuat la:" + outStream);
    System.out.println("Kich thuoc cua dong xuat la:" + outStream.size());
    // tạo 1 dòng nhập từ  buffer của dòng xuất
    ByteArrayInputStream inStream;
    inStream = new ByteArrayInputStream(outStream.toByteArray());
    int Count = inStream.available();
    System.out.println("Kich thuoc cua dong nhap la:" + Count);
    // tạo 1 biến byte array
    byte [] SS = new byte[Count];
    // Đọc từ dòng nhập ra mảng byte SS
    int ReadCount= inStream.read(SS,0,Count);
    System.out.println("So byte doc duoc:" + ReadCount + " bytes");
    System.out.println("Noi dung doc duoc:" + new String(SS));
  }
}
```

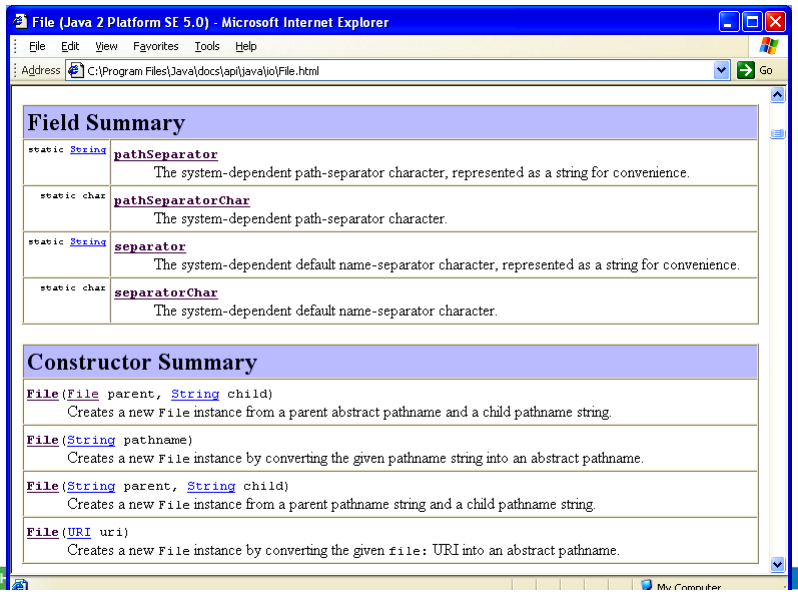
Kết quả

```
Dong xuat la:Ve ve ve, mua he sang
Kich thuoc cua dong xuat la:21
Kich thuoc cua dong nhap la:21
So byte doc duoc:21 bytes
Noi dung doc duoc:Ve ve ve, mua he sang
```

3.3.3- Lớp File

- Giúp truy xuất thuộc tính file và thư mục.
- Bao gói các đối tượng file của hệ thống máy chủ, giúp truy xuất hệ thống thư mục tập tin: Tạo/xóa thư mục-tập tin, truy xuất các thuộc tính file…

Lớp File....



Lớp File...

File (Java 2 Platform SE 5.0) - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Address C:\Program Files\Java\docs\api\java\io\File.html

Method Summary

boolean	**canRead**() Tests whether the application can read the file denoted by this abstract pathname.
boolean	**canWrite**() Tests whether the application can modify the file denoted by this abstract pathname.
int	**compareTo**(File pathname) Compares two abstract pathnames lexicographically.
boolean	**createNewFile**() Atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist.
static File	**createTempFile**(String prefix, String suffix) Creates an empty file in the default temporary-file directory, using the given prefix and suffix to generate its name.
static File	**createTempFile**(String prefix, String suffix, File directory) Creates a new empty file in the specified directory, using the given prefix and suffix strings to generate its name.
boolean	**delete**() Deletes the file or directory denoted by this abstract pathname.
void	**deleteOnExit**() Requests that the file or directory denoted by this abstract pathname be deleted when the virtual machine terminates.
boolean	**equals**(Object obj) Tests this abstract pathname for equality with the given object.

My Computer

Minh họa truy xuất thuộc tính File

f1 - Notepad

File Edit Format Help

Day la noi dung tap tin f1.txt

```
//FileDemo.java
import java.io.*;
import java.util.Date;
class FileDemo
{ public static void main (String args[]) throws IOException
  { File f = new File("f1.txt");
    System.out.println("Ten file la:" + f.getName());
    System.out.println("Ten file tuyet doi la:" + f.getAbsoluteFile());
    System.out.println("Duong dan tuyet doi la:" + f.getAbsolutePath());
    System.out.println("Path chuan la:" + f.getCanonicalPath());
    System.out.println("Ngay cap nhat cuoi cung la:" + new Date(f.lastModified()));
    System.out.println("Thuoc tinh Hidden:" + f.isHidden());
    System.out.println("Thuoc tinh can-read:" + f.canRead());
    System.out.println("Thuoc tinh can-write:" + f.canWrite());
    System.out.println("Kich thuoc:" + f.length() + " bytes");
  }
}
```

C:\PROGRA~1\XINOXS~1\JCREAT~2\GE2001.exe

```
Ten file la:f1.txt
Ten file tuyet doi la:E:\TaiLieuCacMonHocTuSoan\Java\Java-CoBan\BtCh10-IO\f1.txt

Duong dan tuyet doi la:E:\TaiLieuCacMonHocTuSoan\Java\Java-CoBan\BtCh10-IO\f1.tx
t
Path chuan la:E:\TaiLieuCacMonHocTuSoan\Java\Java-CoBan\BtCh10-IO\f1.txt
Ngay cap nhat cuoi cung la:Mon Jan 03 20:43:20 PST 2005
Thuoc tinh Hidden:false
Thuoc tinh can-read:true
Thuoc tinh can-write:true
Kich thuoc:30 bytes
Press any key to continue...
```

Hành vi lastModified() trả về 1 số long mô tả chênh lệnh mili giây kể từ January 1, 1970, 00:00:00 GMT. Thông qua 1 đối tượng Date đổi chênh lệch mili giây này trở lại thành ngày giờ GMT

Minh họa truy xuất thư mục

FileDemo2.java

```
//FileDemo2.java
import java.io.*;
import java.util.Date;
class FileDemo2
{ public static void main (String args[]) throws IOException
  { File f = new File("../BtCh10-IO");
    String S = f.isDirectory() ? "Thu muc" : "Tap tin";
    System.out.println("../BtCh10-IO la:" + S);
    String L[]= f.list();
    System.out.println("Noi dung thu muc:");
    for (int i=0;i<L.length;++i)
    {  File f2 = new File (f,L[i]);
       System.out.println(L[i] +"   " + (f2.isFile()? "Tap tin" : "Thu muc"));
    }
  }
}
```

C:\PROGRA~1\XINOXS~1\JCREAT~2\GE2001.exe

```
../BtCh10-IO la:Thu muc
Noi dung thu muc:
ByteArrayDemo.class  Tap tin
ByteArrayDemo.java  Tap tin
Data1.txt  Tap tin
DataInputOutputStreamDemo.class  Tap tin
DataInputOutputStreamDemo.java  Tap tin
DSSACH.class  Tap tin
f1.txt  Tap tin
f2.txt  Tap tin
FileDemo.class  Tap tin
FileDemo.java  Tap tin
FileDemo2.class  Tap tin
FileDemo2.java  Tap tin
FileInputOutputStreamDemo.class  Tap tin
FileInputOutputStreamDemo.java  Tap tin
File_1.class  Tap tin
File_1.java  Tap tin
File_2.class  Tap tin
File_2.java  Tap tin
File_3.class  Tap tin
File_3.java  Tap tin
File_4.java  Tap tin
IntMatrix.class  Tap tin
IntMatrix.java  Tap tin
```

../ chỉ thị cho thư mục cha của thư mục hiện hành

3.3.4- Lớp FileInputStream, FileOutputStream

- Là các lớp cho việc đọc/ghi file theo từng byte vật lý.
- Lớp FileInputStream là con của lớp InputStream
- Lớp FileOutputStream là con của lớp OutputStream

Lớp FileInputStream

- Constructors

FileInputStream (File f) // f đã có

FileInputStream (FileDescriptor fdesc)

FileInputStream (String FileName)

- Methods

Ngoài những methods được override từ các phương thức của lớp cha InputStream (read(...),...), có 2 hành vi được thêm vào:

protected void **finalize()** **throws IOException**: Đóng dòng (file)

FileDescriptor **getFD()** : Lấy file descriptor kết nối với file thực mà đối tượng FileInputStream này sử dụng.

Lớp FileOutputStream

- Constructors

FileOutputStream (File f) // f đã có

FileOutputStream (File f, boolean append) // f đã có

FileOutputStream (FileDescriptor fdesc) // fdesc đã có

FileOutputStream (String FileName)

FileOutputStream (String FileName, boolean append)

- Methods

Ngoài những methods được override từ các phương thức của lớp cha OutputStream, có 2 hành vi được thêm vào:

protected void **finalize()** **throws IOException** : Đóng dòng (file)

FileDescriptor **getFD()** : Lấy file descriptor kết nối với file thực mà đối tượng FileOutputStream này sử dụng.

Minh họa

FileInputOutputStreamDemo.java

```
1  //FileInputOutputStreamDemo.java
2  import java.io.*;
3  class FileInputOutputStreamDemo
4  { public static void main(String args[]) throws java.io.IOException
5    { final char BLANK=32;
6      char a[] = {'1','2','3','4','5'};
7      String S="Hello";
8      // Ghi du lieu len file f2.txt
9      FileOutputStream outf = new FileOutputStream("f2.txt");
10     int i;
11     for (i=0;i<5;i++)
12     { outf.write(a[i]);
13       outf.write(BLANK);
14     }
15     for (i=0;i<S.length();++i) outf.write(S.charAt(i));
16     outf.close();
17     // Doc du lieu tu file
18     FileInputStream inf = new FileInputStream("f2.txt");
19     int Count = inf.available();
20     System.out.println("So byte trong file:" + Count);
21     byte buffer[]= new byte[Count];
22     int ReadCount= inf.read(buffer,0,Count);
23     System.out.println("Noi dung file:");
24     System.out.println(new String(buffer));
25     System.out.println("So byte doc duoc:" + ReadCount);
26     inf.close();
27   }
28 }
```

C:\PROGRA~1\XINOXS~1\JCREAT~2...

```
So byte trong file:15
Noi dung file:
1 2 3 4 5 Hello
So byte doc duoc:15
Press any key to continue...
```

BtCh10-IO

Folders: BtCh08-App, BtCh09-Thr, BtCh10-IO, HelloWorld, HocSinh, JPadProHello, NgonNguC

ByteArrayDemo.class, ByteArrayDemo.java, Data1.txt, DataInputOutputStreamDemo.class, DataInputOutputStreamDemo.java, DSSACH.class, f1.txt, f2.txt

Address: E:\TaiLieuCacMonHocTuSoan\Jav

Type: Text Doc 15 bytes

f2.txt - Notepad

1 2 3 4 5 Hello

3.3.5- Lớp RandomAccessFile

- Cung cấp khả năng di chuyển tới lui trong file vì xem đơn vị lưu trữ trong file là byte. Do vậy có thể đọc/ghi file tại những vị trí đã được chỉ định (nên gọi là random access).
- Lớp RandomAccessFile cung cấp cả 2 tác vụ đọc/ghi dữ liệu. Do vậy lớp này có thể dùng để đọc/ghi các dữ liệu thuộc kiểu cơ bản.
- Có các hành vi readXXX(), writeXXX() để đọc ghi các dữ liệu thuộc kiểu cơ bản.

Lớp RandomAccessFile...

- Constructors

RandomAccessFile(File f, String mode)
RandomAccessFile(String Filename, String mode)

mode	Ý nghĩa
"r"	Mở file chỉ để đọc , có thể gây lỗi IOException.
"rw"	Mở file đổ đọc + ghi. Nếu file chưa có thì sẽ tạo mới file này.
"rws" synchronized	Mở file để đọc + ghi. Nếu có hiệu chỉnh nội dung hay dữ liệu mô tả file (metadata) thì đòi hỏi phải được ghi đồng bộ.
"rwd"	Mở file để đọc + ghi và mỗi khi có hiệu chỉnh nội dung file thì phải được đồng bộ.

Lớp RandomAccessFile...

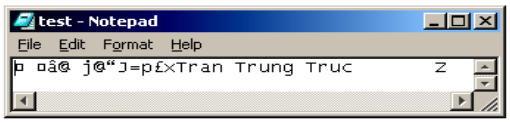
RandomAccessFile Method Summary	
void	**close**() – Đóng file.
FileChannel	**getChannel**() Lấy đối tượng FileChannel kết hợp với .
FileDescriptor	**getFD**() Lấy FileDescriptor.
long	**getFilePointer**() - Lấy vị trí truy xuất hiện hành.
long	**length**() - Lấy đ9ộ dài file.
int void	**read**() Đọc 1 byte. **write(int b)** ghi 1 byte vào vị trí hiện hành
int void	**read**(byte[] b) Đọc 1 mảng byte- trả về số byte đọc được. **write**(byte[] b) ghi mảng b
int void	**read**(byte[] b, int off, int len) **write**(byte[] b, int off, int len) Đọc/ghi len byte đưa vào mảng b từ vị trí off, trả về số byte đọc được.
boolean void	**readBoolean**() Đọc/ghi 1 trị boolean. **writeBoolean** (boolean v)
byte void	**readByte**() Đọc.ghi 1 trị 8 bit có dấu. **writeByte** (int x)
char void	**readChar**() - Đọc 1 ký tự Unicode . **writeChar**(int v) ghi 1 ký tự 2 byte, byte cao ghi trước
double void	**readDouble**() Đọc/ ghi 1 trị double. **writeDouble** (double x) – ghi dạng longbits 8 bytes

Lớp RandomAccessFile...

float void	**readFloat**() Đọc/ ghi 1 trị float. **writeFloat**(float x) ghi dạng intbit 4 bytes
void	**readFully**(byte[] b) Đọc ra cả 1 mảng byte từ vị trí hiện hành.
void	**readFully**(byte[] b, int off, int len) Đọc len byte từ vị trí hiện hành đưa vào vị trí off của mảng b.
int void	**readInt**() Đọc 1 số integercó dấu 32 bit . **writeInt**(int x)
String	**readLine**() Đọc 1 dòng kể từ vị trí hiện hành.
long void	**readLong**() Đọc 1 số long có dấu (64 bit). **writeLong**(long x) , 8 byte, byte cao ghi trước
short void	**readShort**() Đọc 1 số short có dấu (16 bit). **writeShort**(short x)
int	**readUnsignedByte**() Đọc 1 byte không dấu (8 bit).
int	**readUnsignedShort**() Đọc 1 số short không dấu (16 bit).
String void	**readUTF**() Đọc 1 chuỗi. **writeUTF**(String S) ghi chuỗi dạng mã UTF-8 độc lập máy chủ
void	**seek**(long pos) Nhảy đến vị trí pos từ đầu file.
void	**setLength**(long newLength) ấn định độ dài file.
int	**skipBytes**(int n) Bỏ qua n bytes.
void	**writeBytes**(String s) Ghi chuỗi dạng nhóm các bytes.
void	**writeChars**(String s) Ghi chuỗi dạng nhóm ký tự.

Minh họa lớp RandomAccessFile

```
// File_2.java
import java.io.*;
class File_2
{  public static void main(String args[]) throws IOException
   {  RandomAccessFile file = new RandomAccessFile("test.txt","rw");
      file.writeBoolean(true);
      file.writeInt(123456);
      file.writeChar('j');
      file.writeDouble(1234.56);
      String S= "Tran Trung Truc";
      file.writeBytes(S);
      file.writeLong(90);
      file.seek(1);        // Nhẩy về vị trí 1
      System.out.println(file.readInt());
      System.out.println(file.readChar());
      System.out.println(file.readDouble());
      byte s[] = new byte[15];
      file.read(s);  // đọc 15 byte cho phần tên
      String Str= new String (s,"UTF-8"); // đổi ra chuỗi UTF-8
      System.out.println(Str);
      System.out.println(file.readLong());
      file.seek(0); //về vị trí đầu file
      System.out.println(file.readBoolean());
      file.close();
   }
}
```




```
123456
j
1234.56
Tran Trung Truc
90
true
```

Minh họa lớp RandomAccessFile

- **Chú ý:**

(1) Với file đối tượng RandomAccessFile, chúng ta có thể di chuyển tự do trong file. Do vậy để thao tác đọc ghi đúng dữ liệu , chúng ta phải biết rõ trật tự ghi dữ liệu lên file để khi đọc dữ liệu từ file ra biến đúng cách. Trong ví dụ trên:

- **Qúa trình ghi:**

Đầu tiên : Ghi 1 trị boolean (vị trí 0)
Sau đó: Ghi 1 trị int (vị trí 1)
Sau đó: Ghi 1 ký tự.
Sau đó: Ghi 1 trị double
Sau đó : ghi chuỗi S="Tran Trung Truc" , 15 ký tự
Sau đó: Ghi 1 trị long (90)
Nhờ vậy, qúa trình đọc file ra biến biết chỗ để nhảy đến (xem seek(1), seek(0) trong code).

(2) Khi xem file với Notepad của Windows, có những nội dung số ta không đọc được vì Nodepad xem các byte lưu trữ là mã ASCII của ký tự. Chỉ khi đọc bằng code Java rồi xuất ta mới biết rõ nội dung (xem lại kết quả chương trình).

3.3.6- FilterInputStream và FilterOutputStream

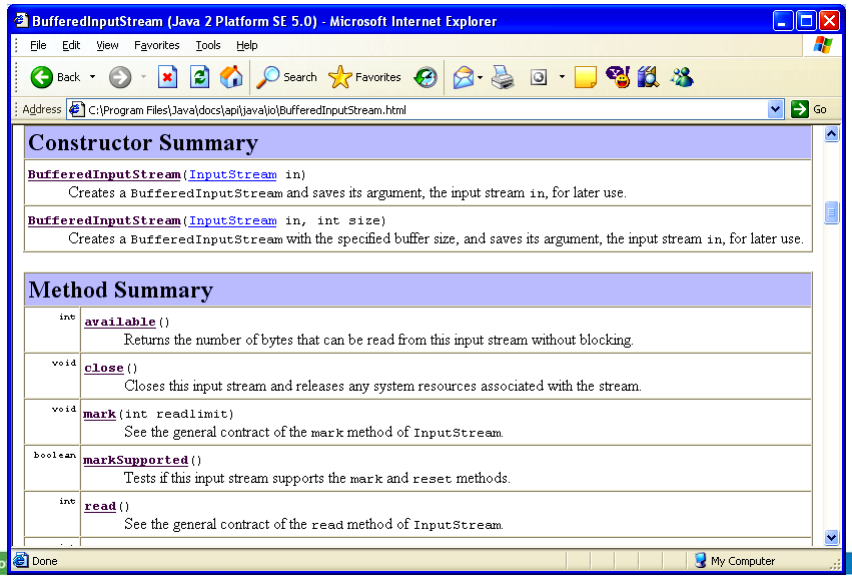
- Là các lớp con của các lớp InputStream và OutputStream tương ứng đảm nhiệm công việc nhập xuất có lọc dữ liệu (nhập xuất có điều kiện)
- Là các lớp cha của các lớp dòng nhập xuất có bộ lọc khác

- java.io.InputStream (implements java.io.Closeable)
  - java.io.ByteArrayInputStream
  - java.io.FileInputStream
  - → java.io.FilterInputStream
    - java.io.BufferedInputStream
    - java.io.DataInputStream (implements java.io.DataInput)
    - java.io.LineNumberInputStream
    - java.io.PushbackInputStream
- java.io.OutputStream (implements java.io.Closeable, java.io.Flushable)
  - java.io.ByteArrayOutputStream
  - java.io.FileOutputStream
  - → java.io.FilterOutputStream
    - java.io.BufferedOutputStream
    - java.io.DataOutputStream (implements java.io.DataOutput)
    - java.io.PrintStream (implements java.lang.Appendable, java.io.Closeable)

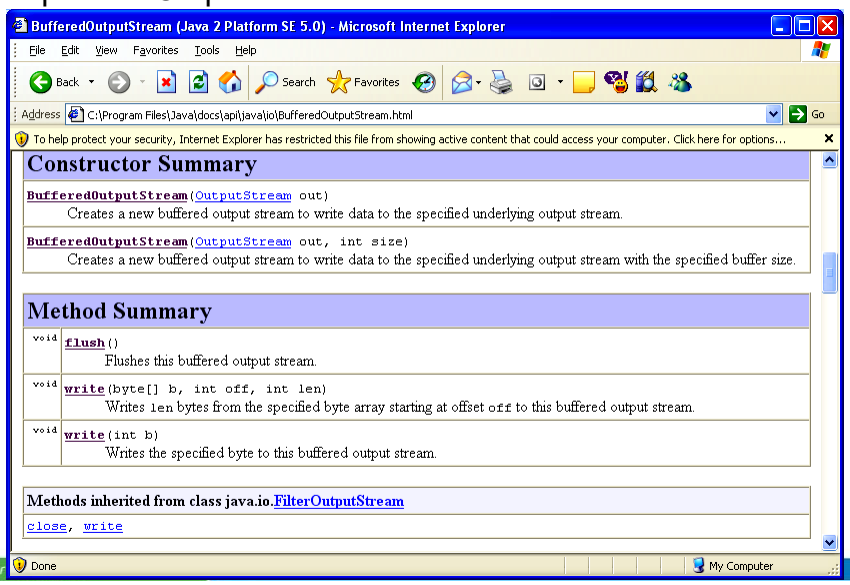
3.3.7- BufferedInputStream và BufferedOutputStream

- Buffer: Bộ nhớ đệm của qúa trình đọc ghi dữ liệu với các dòng nhập xuất nhằm tăng hiệu qủa quá trình đọc ghi dữ liệu (đọc ghi theo khối lớn thay vì theo từng byte). Chúng ta có thể lấy dữ liệu từ buffer thay vì từ nguồn dữ liệu.
- Đây là hai lớp quản lý nhập xuất dữ liệu có đệm .
- Bàn phím là 1 thiết bị nhập có đệm.
- Màn hình là 1 thiết bị xuất có đệm.
- Lớp tự hiện thực dòng nhập xuất chuẩn (bàn phím, màn hình) thường là lớp con của 2 lớp này.

Lớp BufferInputStream

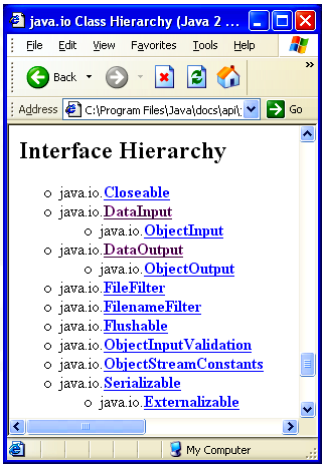


Lớp BufferOutputStream

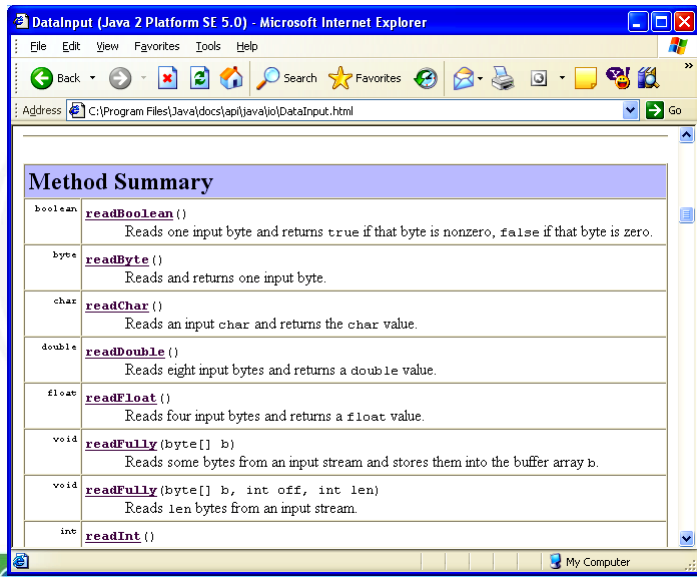


3.3.8- DataInput interface và DataOutput interface DataInputStream và DataOutputStream

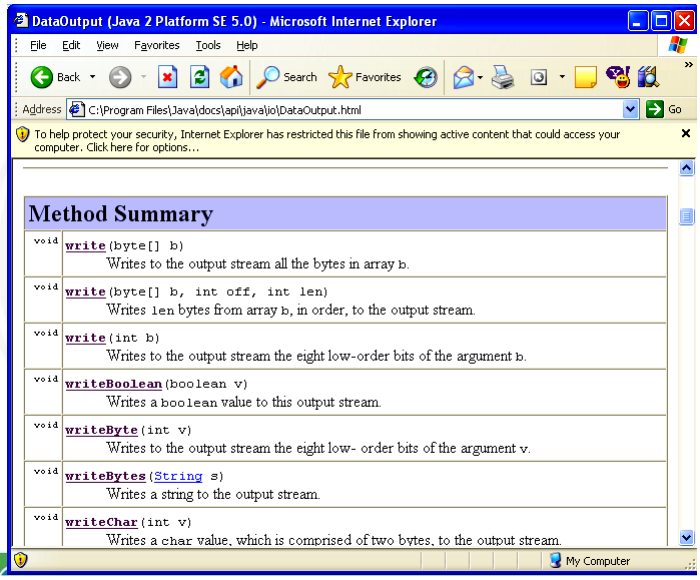
- DataInput interface được dùng để đọc các byte nhị phân từ 1 dòng byte vật lý (InputStream) và xây dựng lại các byte này thành các dữ liệu có kiểu cơ bản (primitive data types). Inteface này cũng chuyển đổi chuỗi dạng UTF-8 có sửa đổi thành dữ liệu dạng String .
- DataOutput interface lại làm ngược lại những gì mà DataInput interface đã làm.
- Hai interface này được 2 lớp **DataInputStream** và **DataOutputStream** hiện thực.



DataInput interface



DataOutput interface

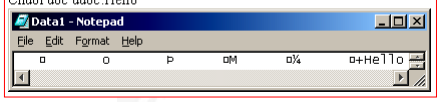


Minh họa về sử dụng DataInputStream và DataOutputStream

```
import java.io.*;  //DataInputOutputStreamDemo.java
class DataInputOutputStreamDemo
{ public static void main(String args[])
  { int n=5;
    long a[] = {111,222,333,444,555 };
    String S="Hello"; int i;
    try  // Ghi du lieu len file xuat outfile
    { OutputStream outfile = new FileOutputStream("Data1.txt");
      DataOutputStream OutStream = new DataOutputStream(outfile);
      OutStream.writeInt(n);
      for (i=0;i<n;i++) OutStream.writeLong(a[i]);
      OutStream.writeBytes(S);  // hoặc OutStream.writeChars(S);
      OutStream.close(); outfile.close();
    }
    catch (Exception e) { System.out.println("Error 1:" + e); }
    try    // Doc du lieu tu file
    { InputStream infile = new FileInputStream("Data1.txt");
      DataInputStream InStream = new DataInputStream(infile);
      n= InStream.readInt();
      long a2[] = new long[n];
      for (i=0;i<n;i++) a2[i]=InStream.readLong();
      byte S2[]= new byte [5];                    // hoặc   char S2[]= new char [5]
      int Count =InStream.read(S2,0,5);    // for (i=0;i<n;++i) S2[i]=InStream.readChar();
      System.out.println("Mang doc duoc:");      // Xuat du lieu doc duoc
      for (i=0;i<n;++i) System.out.print(a2[i]+ " ");
      System.out.println("\n" + "Chuoi doc duoc:" + new String(S2));
      InStream.close();  infile.close();
    }
    catch (Exception e) { System.out.println("Error 2:" + e);}
  }
}
```

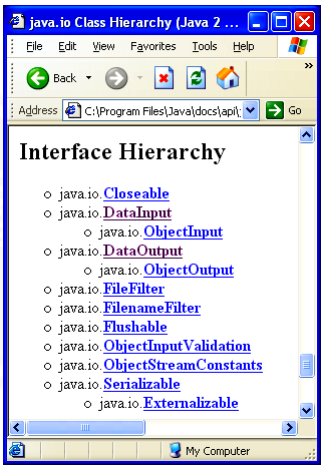
Kết qủa

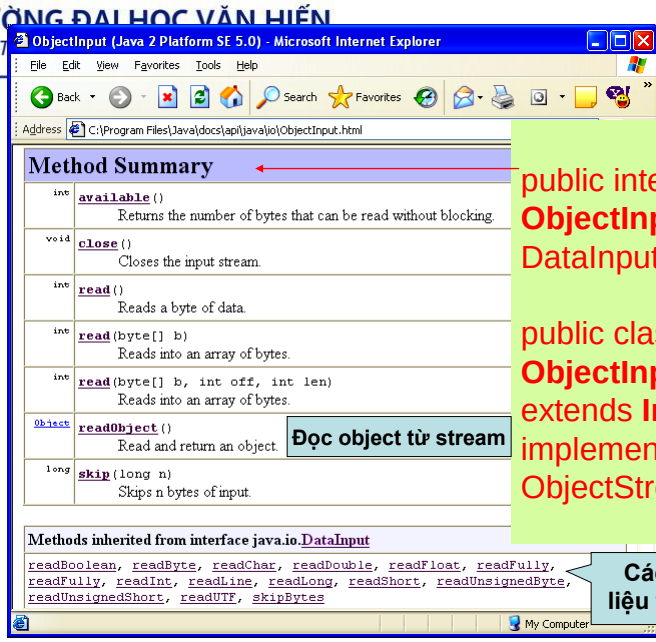
Mang doc duoc:
111 222 333 444 555
Chuoi doc duoc:Hello



3.3.9- Interface ObjectInput và ObjectOutput

- Là 2 interface con của DataInput và DataOutput interfaces cho việc nhập xuất đối tượng.
- Hai lớp **ObjectInputStream** và **ObjectOutputStream** hiện thực 2 interface này.



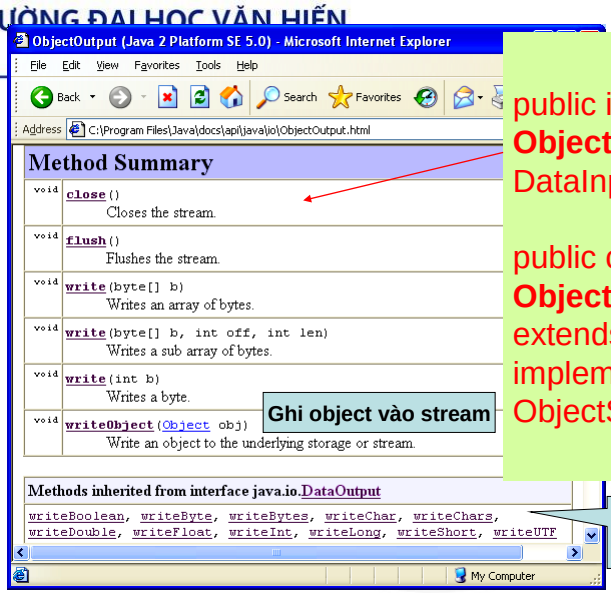


Đọc object từ stream

public interface **ObjectInput** extends DataInput

public class **ObjectInputStream** extends **InputStream** implements ObjectInput, ObjectStreamConstants

Các hành vi đọc dữ liệu thuộc kiểu cơ bản



Ghi object vào stream

public interface **ObjectOutput** extends DataInput

public class **ObjectOutputStream** extends **OutputStream** implements ObjectOutput, ObjectStreamConstants

Các hành vi ghi dữ liệu thuộc kiểu cơ bản

Để đọc ghi object với stream

- class của object phải implements interface Serializable để thống nhất cách đọc ghi object của hai tác vụ readObject và writeObject.
- Việc đồng bộ cơ chế đọc ghi đối tượng thông qua hành vi toString(), tuần tự hóa dữ liệu của đối tượng.

Minh họa ObjectInputStream và ObjectOutputStream

Lớp mô tả cho 1 cuốn sách

```
1   //File_3.java
2   import java.io.*;
3   class SACH  implements Serializable
4   { final int MaSachL = 5;
5     final int TuaSachL = 25;
6     final int GiaL  = 7;
7     String MaSach = new String (new char[MaSachL]);
8     String TuaSach = new String(new char[TuaSachL]);
9     int Gia =0;
10    SACH() { MaSach=""; TuaSach=""; Gia=0;}
11    SACH (String m, String t, int g)
12    { MaSach=m; TuaSach=t; Gia = g;}
13    String getMaSach() { return MaSach;}
14    String getTuaSach() { return TuaSach;}
15    int getGia() { return Gia;}
16    String ThemDuoi(String s, int Length)
17    { String ss= s;
18      while (ss.length()<Length) ss = ss + " ";
19      return ss;
20    }
21    String ThemDau(int x, int Length)
22    { String ss= Integer.toString(x);
23      while (ss.length()<Length) ss = " " + ss;
24      return ss;
25    }
26    public void Xuat()
27    { String S="";
28      S= ThemDuoi(MaSach,MaSachL) + ThemDuoi(TuaSach,TuaSachL) +
29         ThemDau(Gia,GiaL);
30      System.out.println(S);
31    }
32    public String toString()
33    { String s= MaSach +" " + TuaSach + Integer.toString(Gia);
34      return s;
35    }
36  }
```

implements Serializable để đồng bộ 2 quá trình đọc ghi đối tượng với stream

Các tác vụ chỉnh dạng giúp xuất dữ liệu cho đẹp

Minh họa ObjectInputStream và ObjectOutputStream

```
37 class DSSACH
38 { int MaxN=100;
39   int n;
40   SACH ds[] = new SACH[MaxN];
41   DSSACH() { n=0; }
42   boolean Exist(String ms)
43   { for (int i=0;i<n;++i) if (ms==ds[i].getMaSach()) return true;
44     return false;
45   }
46   void Add (SACH s)
47   { if (Exist(s.getMaSach())) return;
48     ds[n++]=s;
49   }
50   void Add (String m, String t, int g)
51     { if (Exist(m)) return;
52       ds[n++]= new SACH(m,t,g);
53     }
54   void AddFromFile(String Filename) throws IOException
55   { FileInputStream file = new FileInputStream(new File(Filename));
56     ObjectInputStream f= new ObjectInputStream (file);
57     boolean eof=false;
58     do
59     {  try
60         { SACH s=(SACH)f.readObject();
61           if (s!=null) this.Add(s);
62         }
63        catch (Exception e)
64         { eof=true;
65         }
66     }
67     while (!eof);
68     f.close();
69     file.close();
70   }
71   void WriteToFile(String Filename) throws IOException
72   { FileOutputStream file = new FileOutputStream(new File(Filename));
73     ObjectOutputStream f= new ObjectOutputStream (file);
74     for (int i=0;i<n;++i) f.writeObject(ds[i]);
75     f.close();
76     file.close();
77   }
78   void Xuat()
79   { for (int i=0;i<n;++i) ds[i].Xuat();
80   }
```

- Lớp mô tả cho 1 tập các cuốn sách
- Kiểm tra sự tồn tại của 1 sách với mã số đã biết
- Thêm 1 cuốn sách
- Thêm các cuốn sách từ 1 file
- Ghi tập các cuốn sách lên 1 file
- Xuất tập các cuốn sách ra màn hình

Minh họa ObjectInputStream và ObjectOutputStream

Lớp cho chương trình chính

```
82 class QUANLYSACH
83 {
84   public static void main(String args[])throws IOException
85   { DSSACH ds = new DSSACH();
86     ds.Add("S001", "Java co ban", 18000);
87     ds.Add("S002", "Java nang cao", 24000);
88     System.out.println("Danh sach truoc khi ghi file");
89     ds.Xuat();
90     ds.WriteToFile("Sach1.txt");
91     DSSACH ds2= new DSSACH();
92     ds2.AddFromFile("Sach1.txt");
93     System.out.println("Danh sach doc tu file:");
94     ds2.Xuat();
95   }
96 }
```

Kết quả

```
C:\PROGRA~1\XINOXS~1\JCREAT~2\GE2001.exe
Danh sach truoc khi ghi file
S001 Java co ban              18000
S002 Java nang cao            24000
Danh sach doc tu file:
S001 Java co ban              18000
S002 Java nang cao            24000
Press any key to continue...
```

Nếu bỏ implements Serializable trong class SACH

```
C:\PROGRA~1\XINOXS~1\JCREAT~2\GE2001.exe
Danh sach truoc khi ghi file
S001 Java co ban              18000
S002 Java nang cao            24000
Exception in thread "main" java.io.NotSerializableException: SACH
        at java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1075

        at java.io.ObjectOutputStream.writeObject(ObjectOutputStream.java:291)
        at DSSACH.WriteToFile(File_3.java:74)
        at QUANLYSACH.main(File_3.java:90)
Press any key to continue...
```

3.3.10- Các dòng trừu tượng cho ký tự Reader , Writer

- Đơn vị xử lý trong dòng là ký tự → 2 byte.

public abstract class **Reader** extends Object
implements Readable, Closeable
public abstract class **Writer** extends Object
implements Appendable, Closeable, Flushable

- Các lớp con của Writer chỉ cần override write(char[], int, int), flush(), và close()
- Các lớp con của Reader chỉ cần override read(char[], int, int) và close().
- Có thể overide thêm các methods khác hoặc thêm methods nếu muốn.

Các lớp dẫn xuất

- java.io.Reader (implements java.io.Closeable, java.lang.Readable)
  - java.io.BufferedReader
    - java.io.LineNumberReader
  - java.io.CharArrayReader
  - java.io.FilterReader
    - java.io.PushbackReader
  - java.io.InputStreamReader
    - java.io.FileReader
  - java.io.PipedReader
  - java.io.StringReader

- java.io.Writer (implements java.lang.Appendable, java.io.Closeable, java.io.Flushable)
  - java.io.BufferedWriter
  - java.io.CharArrayWriter
  - java.io.FilterWriter
  - java.io.OutputStreamWriter
    - java.io.FileWriter
  - java.io.PipedWriter
  - java.io.PrintWriter
  - java.io.StringWriter

Reader class

class Reader Summary - throws IOException	
protected Object	**lock** : Field được dùng để đồng bộ các tác vụ của dòng
protected	**Reader()** , contructor tạo 1 dòng nhập ký tự có cơ chế đồng bộ
protected	**Reader(Object lock)** : Tạo dòng nhập với đối tượng đồng bộ đã có
abstract void	**close()** – Đóng dòng
void	**mark(int readAheadLimit)** – Đánh dấu vị trí hiện hành
boolean	**markSupported()** : Kiểm tra dòng có hỗ trợ tác vụ đánh dấu?
int	**read()** - Đọc 1 ký tự từ dòng
int	**read(char[] cbuf)** : Đọc 1 mảng ký tự, trả về đọc được (-1 khi khi hết dòng)
abstract int	**read(char[] cbuf, int off, int len)** Đọc len ký tự lưu vào mảng cbuf từ vị trí off .Trả về số ký tự đã đọc được (-1 khi hết dòng)
int	**read(CharBuffer target)** Đọc các ký tự đưa vào buffer, trả về số ký tự đã đọc được (-1 khi hết dòng)
boolean	**ready()** - Kiểm tra dòng có sẵn sàng để đọc hay đang bị khóa
void	**reset()** : Reset dòng về vị trí đánh dấu cuối cùng nếu có
long	**skip(long n)** - bỏ qua n ký tự

class Writer

Class Writer Summary	
protected Object	**lock** (filed) The object used to synchronize operations on this stream.
protected	**Writer()** Create a new character-stream writer whose critical sections will synchronize on the writer itself.
protected	**Writer(Object lock)** Create a new character-stream writer whose critical sections will synchronize on the given object.
Writer	**append(char c)** Appends the specified character to this writer.
Writer	**append(CharSequence csq)** Appends the specified character sequence to this writer.
Writer	**append(CharSequence csq, int start, int end)** Appends a subsequence of the specified character sequence to this writer.
abstract void	**close()** Close the stream, flushing it first.
abstract void	**flush()** Flush the stream.
void	**write(char[] cbuf)**

3.3.11- Lớp BufferedReader và BufferedWriter

- Dòng nhập/xuất ký tự có sử dụng bộ đệm
- Giúp nhập xuất dữ liệu theo dạng ký tự, dòng (line) ký tự.

Lớp BufferedReader

Field Summary

Fields inherited from class java.io.Reader
lock

Constructor Summary

BufferedReader(Reader in)	Create a buffering character-input stream that uses a default-sized input buffer.
BufferedReader(Reader in, int sz)	Create a buffering character-input stream that uses an input buffer of the specified size.

Method Summary

void	close() Close the stream.
void	mark(int readAheadLimit) Mark the present position in the stream.
boolean	markSupported() Tell whether this stream supports the mark() operation, which it does.
int	read() Read a single character.
int	read(char[] cbuf, int off, int len) Read characters into a portion of an array.
String	readLine() Read a line of text.
boolean	ready() Tell whether this stream is ready to be read.
void	reset() Reset the stream to the most recent mark.
long	skip(long n) Skip characters.

Methods inherited from class java.io.Reader
read, read

Lớp BufferedWriter

Field Summary
Fields inherited from class java.io.Writer
lock

Constructor Summary
BufferedWriter(Writer out) Create a buffered character-output stream that uses a default-sized output buffer.
BufferedWriter(Writer out, int sz) Create a new buffered character-output stream that uses an output buffer of the given size.

Method Summary	
void	close() Close the stream.
void	flush() Flush the stream.
void	newLine() Write a line separator.
void	write(char[] cbuf, int off, int len) Write a portion of an array of characters.
void	write(int c) Write a single character.
void	write(String s, int off, int len) Write a portion of a String.

Methods inherited from class java.io.Writer
append, append, append, write, write

Methods inherited from class java.lang.Object
clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

3.3.12- Lớp InputStreamReader
Lớp OutputStreamReader

- Các lớp cầu nối giữa dòng vật lý và dòng ký tự.
- InputStreamReader sẽ đọc các byte vật lý rồi chuyển thành các ký tự.
- OutputStreamWriter sẽ ghi các ký tự thành byte vật lý
- Thường được dùng cho bàn phím và màn hình vì đây là các thiết bị ký tự, khi xuất nhập 1 số (byte vật lý) CẦN CÓ SỰ CHUYỂN ĐỔI BYTE VẬT LÝ ←→ KÝ TỰ

Sử dụng kết hợp để có hiệu qủa

Khai báo dòng nhập là bàn phím:

Gói InputStreamReader trong BufferedReader

```
BufferedReader in = new BufferedReader(new
    InputStream(System.in));
```

Khai báo dòng xuất là màn hình:

Gói OutputStreamWriter trong BufferedWriter

```
Writer out = new BufferedWriter(new
    OutputStreamWriter(System.out));
```


Lớp InputStreamReader

Field Summary

Fields inherited from class java.io.Reader

lock

Constructor Summary

InputStreamReader(InputStream in)	Create an InputStreamReader that uses the default charset.
InputStreamReader(InputStream in, Charset cs)	Create an InputStreamReader that uses the given charset.
InputStreamReader(InputStream in, CharsetDecoder dec)	Create an InputStreamReader that uses the given charset decoder.
InputStreamReader(InputStream in, String charsetName)	Create an InputStreamReader that uses the named charset.

Method Summary

void	close() Close the stream.
String	getEncoding() Return the name of the character encoding being used by this stream.
int	read() Read a single character.
int	read(char[] cbuf,int offset,int length) Read characters into a portion of an array.
boolean	ready() Tell whether this stream is ready to be read.

Methods inherited from class java.io.Reader

mark, markSupported, read, read, reset, skip

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Lớp OutputStreamWriter

Constructor Summary
`OutputStreamWriter(OutputStream out)` Create an OutputStreamWriter that uses the default character encoding.
`OutputStreamWriter(OutputStream out, Charset cs)` Create an OutputStreamWriter that uses the given charset.
`OutputStreamWriter(OutputStream out, CharsetEncoder enc)` Create an OutputStreamWriter that uses the given charset encoder.
`OutputStreamWriter(OutputStream out, String charsetName)` Create an OutputStreamWriter that uses the named charset.

Field Summary
Fields inherited from class java.io.Writer
`lock`

Method Summary	
void	`close()` Close the stream.
void	`flush()` Flush the stream.
String	`getEncoding()` Return the name of the character encoding being used by this stream.
void	`write(char[] cbuf, int off, int len)` Write a portion of an array of characters.
void	`write(int c)` Write a single character.
void	`write(String str, int off, int len)` Write a portion of a string.

Methods inherited from class java.io.Writer
`append, append, append, write, write`

Methods inherited from class java.lang.Object
`clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait`

Minh họa tự xây dựng một lớp nhập dữ liệu từ bàn phím (xuất dữ liiệu đã có System.out.print(...)

```
import java.io.*;
class MyIO
{ static int ReadInt  () throws java.io.IOException
  { int n=0;
    BufferedReader Obj = new BufferedReader(new
     InputStreamReader(System.in));
    try
       { n= Integer.parseInt(Obj.readLine());   }
    catch (Exception e)
       { System.err.println(e.toString());  System.exit(0); }
    return n;
  }
  < Tương tự cho các tác vụ nhập số long, float,double>
```

```
static char ReadChar  () throws java.io.IOException
 { char x=0;
   Reader Obj = new InputStreamReader(System.in);
   try  { x= (char) Obj.read();   }
   catch (Exception e) { System.err.println(e.toString());  System.exit(0);  }
   return x;
 }

 static int ReadCharCode  () throws java.io.IOException
 { return System.in.read();  }

static String ReadString () throws java.io.IOException
 {  String s;
    BufferedReader Obj = new BufferedReader(new
                                    InputStreamReader(System.in));
    s=Obj.readLine();
    while (s.length()==0) s=Obj.readLine();
    return s;
 }
}
```


3.3.13- Lớp FileReader và FileWriter

- Là các lớp con của lớp InputStreamReader và OutputStreamWriter tương ứng nên thừa kế cac1 khả năng về chuyển đổi ký tự ←→ byte
- Giúp truy cập file

Lớp FileReader

Field Summary
Fields inherited from class java.io.Reader
lock

Constructor Summary
FileReader(File file) Creates a new FileReader, given the File to read from.
FileReader(FileDescriptor fd) Creates a new FileReader, given the FileDescriptor to read from.
FileReader(String fileName) Creates a new FileReader, given the name of the file to read from.

Method Summary
Methods inherited from class java.io.InputStreamReader
close, getEncoding, read, read, ready
Methods inherited from class java.io.Reader
mark, markSupported, read, read, reset, skip

Lớp FileWriter

Field Summary
Fields inherited from class java.io.Writer
lock

Constructor Summary
FileWriter(File file) Constructs a FileWriter object given a File object.
FileWriter(File file, boolean append) Constructs a FileWriter object given a File object.
FileWriter(FileDescriptor fd) Constructs a FileWriter object associated with a file descriptor.
FileWriter(String fileName) Constructs a FileWriter object given a file name.
FileWriter(String fileName, boolean append) Constructs a FileWriter object given a file name with a boolean indicating whether or not to append the data written.

Method Summary
Methods inherited from class java.io.OutputStreamWriter
close, flush, getEncoding, write, write, write
Methods inherited from class java.io.Writer
append, append, append, write, write
Methods inherited from class java.lang.Object
clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

3.3.14- Lớp CharArrayReader và Lớp CharArrayWriter

- Hiện thực 1 mảng ký tự (character buffer) cho việc nhập xuất.

public class **CharArrayReader** extends Reader

public class **CharArrayWriter** extends Writer

public class **CharArrayReader** extends Reader

Field Summary

protected char[]	**buf** The character buffer.
protected int	**count** The index of the end of this buffer.
protected int	**markedPos** The position of mark in buffer.
protected int	**pos** The current buffer position.

Constructor Summary

CharArrayReader(char[] buf) Create an CharArrayReader from the specified array of chars.
CharArrayReader(char[] buf, int offset, int length) Create an CharArrayReader from the specified array of chars.

Method Summary

void	**close**() Close the stream.
void	**mark**(int readAheadLimit) Mark the present position in the stream.
boolean	**markSupported**() Tell whether this stream supports the mark() operation, which it does.
int	**read**() Read a single character.
int	**read**(char[] b, int off, int len) Read characters into a portion of an array.
boolean	**ready**() Tell whether this stream is ready to be read.
void	**reset**() Reset the stream to the most recent mark, or to the beginning if it has never been marked.
long	**skip**(long n) Skip characters.

Methods inherited from class java.io.Reader

read, read

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

public class **CharArrayWriter** extends Reader

Field Summary	
protected char[]	**buf** The buffer where data is stored.
protected int	**count** The number of chars in the buffer.

Fields inherited from class java.io.Writer
lock

Constructor Summary
CharArrayWriter() Creates a new CharArrayWriter.
CharArrayWriter(int initialSize) Creates a new CharArrayWriter with the specified initial size.

Method Summary	
CharArrayWriter	**append**(char c) Appends the specified character to this writer.
CharArrayWriter	**append**(CharSequence csq) Appends the specified character sequence to this writer.
CharArrayWriter	**append**(CharSequence csq, int start, int end) Appends a subsequence of the specified character sequence to this writer.
void	**close**() Close the stream.
void	**flush**() Flush the stream.
void	**reset**() Resets the buffer so that you can use it again without throwing away the already allocated buffer.
int	**size**() Returns the current size of the buffer.
char[]	**toCharArray**() Returns a copy of the input data.
String	**toString**() Converts input data to a string.
void	**write**(char[] c, int off, int len) Writes characters to the buffer.
void	**write**(int c) Writes a character to the buffer.
void	**write**(String str, int off, int len) Write a portion of a string to the buffer.
void	**writeTo**(Writer out) Writes the contents of the buffer to another character stream.

3.3.15- Lớp StringReader và StringWriter

- Dòng nhập xuất thông qua String

public class StringReader extends Reader

Field Summary
Fields inherited from class java.io.Reader
lock

Constructor Summary
StringReader(String s) Create a new string reader.

Method Summary	
void	close() Close the stream.
void	mark(int readAheadLimit) Mark the present position in the stream.
boolean	markSupported() Tell whether this stream supports the mark() operation, which it does.
int	read() Read a single character.
int	read(char[] cbuf, int off, int len) Read characters into a portion of an array.
boolean	ready() Tell whether this stream is ready to be read.
void	reset() Reset the stream to the most recent mark, or to the beginning of the string if it has never been marked.
long	skip(long ns) Skips the specified number of characters in the stream.

public class StringWriter extends Writer

Field Summary
Fields inherited from class java.io.Writer
lock

Constructor Summary
StringWriter() Create a new string writer, using the default initial string-buffer size.
StringWriter(int initialSize) Create a new string writer, using the specified initial string-buffer size.

Method Summary	
StringWriter	append(char c) Appends the specified character to this writer.
StringWriter	append(CharSequence csq) Appends the specified character sequence to this writer.
StringWriter	append(CharSequence csq, int start, int end) Appends a subsequence of the specified character sequence to this writer.
void	close() Closing a StringWriter has no effect.
void	flush() Flush the stream.
StringBuffer	getBuffer() Return the string buffer itself.
String	toString() Return the buffer's current value as a string.
void	write(char[] cbuf, int off, int len) Write a portion of an array of characters.
void	write(int c) Write a single character.
void	write(String str) Write a string.
void	write(String str, int off, int len) Write a portion of a string.

3.3.16- public class PrintWriter extends Writer

- Làm nhiệm vụ xuất dữ liệu cơ bản, biểu diễn của đối tượng có định dạng ra một dòng xuất ký tự.

Field Summary	
protected Writer	out The underlying character-output stream of this PrintWriter.

Fields inherited from class java.io.Writer
lock

Constructor Summary
PrintWriter(File file) Creates a new PrintWriter, without automatic line flushing, with the specified file.
PrintWriter(File file, String csn) Creates a new PrintWriter, without automatic line flushing, with the specified file and charset.
PrintWriter(OutputStream out) Create a new PrintWriter, without automatic line flushing, from an existing OutputStream.
PrintWriter(OutputStream out, boolean autoFlush) Create a new PrintWriter from an existing OutputStream.
PrintWriter(String fileName) Creates a new PrintWriter, without automatic line flushing, with the specified file name.
PrintWriter(String fileName, String csn) Creates a new PrintWriter, without automatic line flushing, with the specified file name and charset.
PrintWriter(Writer out) Create a new PrintWriter, without automatic line flushing.
PrintWriter(Writer out, boolean autoFlush) Create a new PrintWriter.

Lớp PrintWriter

Method Summary	
PrintWriter	append(char c) Appends the specified character to this writer.
PrintWriter	append(CharSequence csq) Appends the specified character sequence to this writer.
PrintWriter	append(CharSequence csq, int start, int end) Appends a subsequence of the specified character sequence to this writer.
boolean	checkError() Flush the stream if it's not closed and check its error state.
void	close() Close the stream.
void	flush() Flush the stream.
PrintWriter	format(Locale l, String format, Object... args) Writes a formatted string to this writer using the specified format string and arguments.
PrintWriter	format(String format, Object... args) Writes a formatted string to this writer using the specified format string and arguments.
protected void	setError() Indicate that an error has occurred.
void	write(char[] buf) Write an array of characters.
void	write(char[] buf, int off, int len) Write A Portion of an array of characters.
void	write(int c) Write a single character.
void	write(String s) Write a string.
void	write(String s, int off, int len) Write a portion of a string.

void print (x) ;
void println(x)

3.4- Gói java.lang và lớp System

- Gói java.lang là gói cơ bản của ngôn ngữ Java (*lang*uage).
- Gói này chứa khai báo các lớp wrapper như Boolean, Char, Integer,...
- Có lớp System mô tả cho hệ thống

Gói java.lang

Hierarchy For Package java.lang

Class Hierarchy

- java.lang.**Object**
  - java.lang.**Boolean** (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Character** (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Character.Subset**
    - java.lang.**Character.UnicodeBlock**
  - java.lang.**Class**<T> (implements java.lang.reflect.AnnotatedElement, java.lang.reflect.GenericDeclaration, java.io.Serializable, java.lang.reflect.Type)
  - java.lang.**ClassLoader**
  - java.lang.**Compiler**
  - java.lang.**Enum**<E> (implements java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**Math**
  - java.lang.**Number** (implements java.io.Serializable) ← subclasses:
    - java.lang.**Byte**
    - java.lang.**Double**
    - java.lang.**Float**
    - java.lang.**Integer**
    - java.lang.**Long**
    - java.lang.**Short**
  - java.lang.**Package** (implements java.lang.reflect.AnnotatedElement)
  - java.security.**Permission** (implements java.security.Guard, java.io.Serializable)
  - java.lang.**Process**
  - java.lang.**ProcessBuilder**
  - java.lang.**Runtime**
  - java.lang.**SecurityManager**
  - java.lang.**StackTraceElement** (implements java.io.Serializable)
  - java.lang.**StrictMath**
  - java.lang.**String** (implements java.lang.CharSequence, java.lang.Comparable<T>, java.io.Serializable)
  - java.lang.**StringBuffer** (implements java.lang.CharSequence, java.io.Serializable)
  - java.lang.**StringBuilder** (implements java.lang.CharSequence, java.io.Serializable)
  - java.lang.**System**

public final class **System** extends Object

- **Data**

static PrintStream **err**: mô tả thiết bị xuất lỗi chuẩn (màn hình)

static PrintStream **out**: mô tả thiết bị xuất chuẩn (màn hình)

static PrintStream **in**: mô tả thiết bị nhập chuẩn (bàn phím)

- **Methods**

Các hành vi của lớp System là các hành vi mức hệ thống như truy xuất thời gian, truy xuất các thiết lập hệ thống (trong Config.sys), thoát JVM, nạp 1 lớp trong 1 file, nạp 1 thư viện, kết thúc 1 object, thiết lập thiết bị nhập xuất hệ thống khác với các thiết bị xuất nhập chuẩn, lấy/thiết lập cơ chế bảo mật…

public final class **System** extends Object ....

Các method thường dùng:

static long **currentTimeMillis**() lấy giờ hệ thống theo millisec so với 12 giờ đêm ngày 1/1/1970

static long **nanoTime**() lấy giờ hệ thống theo nanosec (chỉ có từ Java 1.5.0)

static void **exit**(int status) kết thúc JVM

static void **gc(**void) : gọi trình gom rác (garbage collector)

static Properties **getProperties**(void) lấy thiết lập hệ thống (trong Config.sys)

ví dụ về truy xuất thông tin hệ thống

SystemDemo.java

```
1  // SystemDemo.java
2  import java.io.*;
3  import java.util.*;
4  class SystemDemo
5  {  public static void main (String args[])
6     { System.out.print("So giay tu 1/1/1970 :");
7       long t= System.currentTimeMillis();
8       System.out.println(t);
9       System.out.println("Hom nay:" + new Date(t));
10      System.out.println("Mot so bien moi truong Java:");
11      System.out.print("java.runtime.name=");
12      System.out.println(System.getProperty("java.runtime.name"));
13      System.out.print("sun.boot.library.path=");
14      System.out.println(System.getProperty("sun.boot.library.path"));
15    }
16 }
```

Gói java.util chứa lớp Date

```
C:\Program Files\Xinox Software\JCreatorV3\GE2001.exe
So giay tu 1/1/1970 :1131234612939
Hom nay:Sat Nov 05 15:50:12 PST 2005
Mot so bien moi truong Java:
java.runtime.name=Java(TM) 2 Runtime Environment, Standard Edition
sun.boot.library.path=C:\Program Files\Java\jdk1.5.0\jre\bin
Press any key to continue...
```


3.5- ví dụ xuất nhập dữ liệu với tập tin văn bản

- Tự tham khảo trong tài liệu

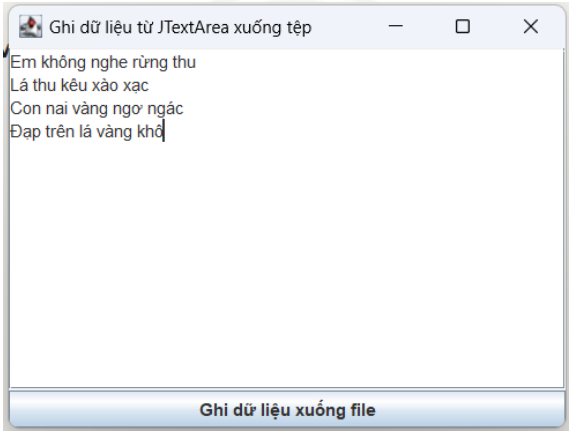
class WriteFile ,

class IntMatrix

Bài tập

1. Viết chương trình đọc dữ liệu từ file Text. Hiển thị dữ liệu lên Form.

2. Viết chương trình nhập dữ liệu từ JtextArea xuống file



3. Viết chương trình sao chép nội dụng từ một tập tin tới một tập tin khác.


```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
public class vidu2 {

    public static void main(String[] args) {
         // Specify the source and destination file paths
        Path sourcePath = Paths.get("source.txt");
        Path destinationPath = Paths.get("destination.txt");

        try {
            // Copy the contents of the source file to the destination file
            Files.copy(sourcePath, destinationPath);
            System.out.println("Đã copy thành công!");
        } catch (IOException e) {
            System.err.println("lỗi không thể copy: " + e.getMessage());
        }

    }

}
```

4. Viết chương trình truy cập ngẫu nhiên tập tin


```java
import java.io.IOException;
import java.io.RandomAccessFile;
public class vidu3 {

    public static void main(String[] args) {
        String fileName = "example.txt";

            try {
                // Create a RandomAccessFile object in read-write mode
                RandomAccessFile raf = new RandomAccessFile(fileName, "rw");

                raf.seek(0); // Move to the beginning of the file
                raf.writeUTF("Hello, World!"); // Write a string

                raf.seek(50); // Move to the 50th byte
                raf.writeInt(12345); // Write an integer

                // Read data from the file at different positions
                raf.seek(0); // Move to the beginning of the file
                String str = raf.readUTF(); // Read the string
                System.out.println("String read from file: " + str);

                raf.seek(50); // Move to the 50th byte
                int num = raf.readInt(); // Read the integer
                System.out.println("Integer read from file: " + num);

                // Close the RandomAccessFile
                raf.close();
            } catch (IOException e) {
                System.err.println("An error occurred: " + e.getMessage());
            }
```

3.6 – Tóm tắt

- Gói java.io chứa các lớp cho việc xuất nhập dữ liệu.
- Các dòng xuất nhập được chia thành 2 loại: dòng văn bản, dòng byte vật lý.
- Dòng văn bản xử lý dữ liệu theo từng ký tự 2 byte
- Dòng byte vật lý xử lý dữ liệu theo từng byte.
- Tác vụ nhập xuất có thể gây lỗi runtime nên cần throws IOException.
- Khi lưu trữ dữ liệu vào dòng, cần chọn 1 định dạng lưu trữ trước để khi phải đọc ra sẽ đọc được đúng dữ liệu.